OpenRec: Bridging Offline Experiments and Interactive Web Recommendation

Zheng Ma
Alibaba
Hangzhou, China
xsank@foxmail.com

ABSTRACT

A Web recommender combines candidate retrieval, feature computation, ranking, presentation rules, and user feedback. Studying these components only through offline model metrics leaves their interaction with a running application difficult to inspect. OpenRec is an open-source system connecting offline recommendation development to an interactive Web application. It combines configurable retrieval and ranking graphs, shared feature definitions with persisted fitted encoders, and feedback ingestion into serving state and historical data. The demonstration lets participants inspect candidate supply and result composition, connect a trained model's feature selection to online scoring, and observe how explicit feedback affects subsequent requests. A management console supports these activities through validated configuration and artifact changes. We describe the implemented recommendation paths, existing cross-component and offline evidence, and a hands-on session using example identities. Runtime interface captures illustrate the interaction, and KuaiRand experiments provide offline evidence for the shared feature representation.

CCS CONCEPTS

• **Information systems** → **Recommender systems**.

KEYWORDS

recommender systems, Web applications, multi-stage recommendation, training-serving consistency, user feedback

1 INTRODUCTION

Offline recommendation experiments produce retrieval scores or rankers; a Web application must also combine channels, filter exposed or disliked items, encode current features, and arrange a list. Feedback then changes later requests and supplies future training data. Model metrics alone cannot show which channels supplied candidates, whether scoring used the fitted representation, or when feedback reached serving state.

OpenRec connects these steps for researchers and developers. A Web page exposes recommendations, scores, and feedback; a console exposes graphs, feature selections, and evaluated artifacts. The 0.1.0 distribution pins compatible component revisions.[1] Its three integrated capabilities are *composable recommendation* across retrieval, ranking, and list policies; *shared feature contracts* linking temporal samples and fitted encoders to serving; and *observable feedback paths* into subsequent requests and training history. We contribute an implemented integration and demonstration, not a new ranker or a measured productivity gain.

Positioning. RecBole focuses on standardized recommendation model implementation and evaluation [6]; the earlier OpenRec framework emphasizes reusable algorithm modules [5]. Our project under the `open-rec` organization is distinct from that framework. The WWW demonstration RecAI organizes LLM-based recommendation tools around application capabilities [2]. Our focus is the connection between offline artifacts and multi-stage, feedback-aware Web recommendation. Model publication is one supporting operation within this workflow; the demonstration also exposes recommendation-specific candidate supply, list policies, feature semantics, and feedback consumption.

2 CONNECTING EXPERIMENTS TO WEB RECOMMENDATION

Figure 1 adapts the project's Cluster architecture into online, streaming, offline, control, and monitoring paths. Figure 2 expands the default item DAG using its actual configuration. The full demonstration uses cluster mode; standalone supports smaller graph experiments and normally bypasses remote ranking.

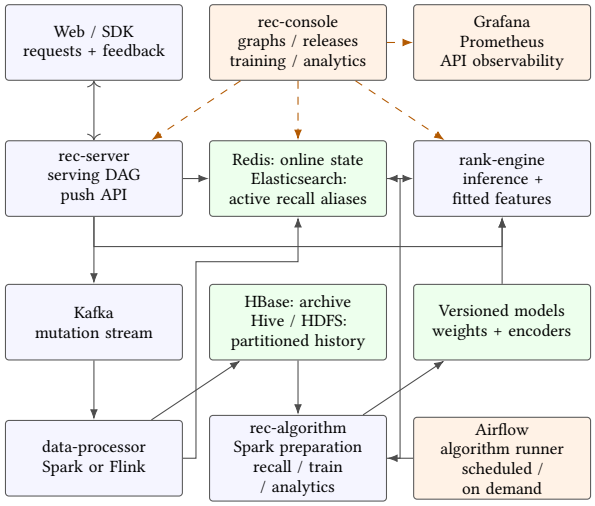
2.1 Candidate Supply and Result Composition

Figure 2 expands the item-serving DAG inside rec-server. Eight retrieval channels feed candidate combination alongside exposure and blacklist inputs; the user-ID and reserved feature nodes precede ranking, followed by a list policy and collection. With `debug=true`, the API reports recall-node status and output counts before filtering and final selection. Result cards show channel labels and distinguish recall from rank scores. These views separate candidate supply from final list membership: an overlapping candidate can be retrieved by several channels but occupy only one result slot.

Beyond enabling channels, developers can configure recall/rank score fusion and post-ranking policies. A channel-allocation rule divides result slots by normalized channel ratios, assigning overlapping candidates to their primary channel and filling shortages by score. An alternative sliding-window rule limits repeated category or tag attributes. Strict constraints can yield fewer results if the candidate pool is insufficient. These are configurable list policies, not guarantees of improved relevance.

[1] Project: https://github.com/open-rec.

WWW Companion '27, May 2027, Dublin, Ireland
2027.



Solid: selected API/data paths. Dashed: console control/access.
Console also controls Airflow; Prometheus scrapes API metrics.
Offline writes staging indexes; console activates serving aliases.

Figure 1: Cluster architecture adapted from the project profile: online serving, feedback, offline computation, control, analytics, and monitoring. Console publication connects staged artifacts to serving.

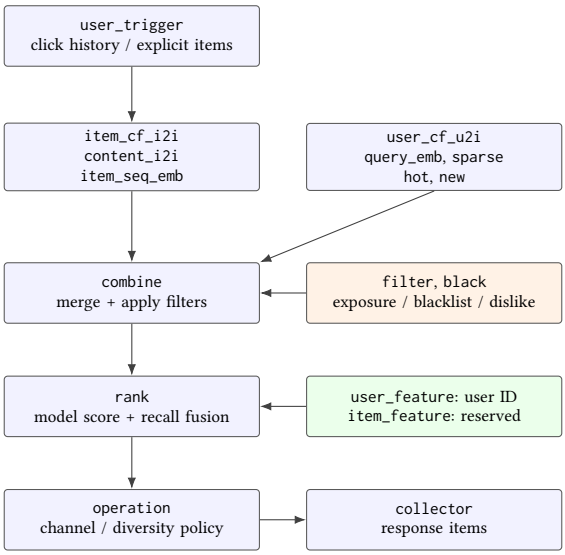


Figure 2: Configured item DAG: 17 nodes, 18 dependencies. Arrows apply to every named node in a group. User-feature supplies the user ID; item-feature is a reserved hook. Actual encoding runs in rank-engine.

Graph changes apply to named experiments selected through `params.ab`. Validation checks supported nodes, typed configuration, and dependencies; requests retain their selected graph snapshot. Readiness probes verify representative execution after publication. The console retains versions for restoration. Neither statistical A/B analysis nor candidate-level causal explanations are provided.

2.2 From Feature Definitions to Online Scores

OpenRec separates four artifacts: canonical logical feature definitions, a model's ordered feature selection, fitted encoders, and materialized feature values. The catalog describes semantics such as event identity, aggregation windows, time boundaries, and online/offline availability. Offline pandas/Spark implementations and the streaming feature core implement these contracts; shared fixtures check selected behaviors across implementations. Cluster training prepares point-in-time samples from mutation history so that later events and entity updates are not simply joined into earlier labels.

The console offers supported LR, FM, and LightGBM feature selections and triggers offline training through Airflow. Training produces evaluated releases; it does not update the live model. Each release binds weights to selected feature definitions, fitted vocabularies/scaling, input dimensions, and checksums. The ranking service imports the algorithm library's encoding contracts and loads the retained fitted representation instead of fitting a new encoder on serving data. It checks selected definitions and materialized columns before activation and uses a consistent model/feature snapshot for a score request.

Deployment supports a narrower feature subset than the research code. Catalog membership does not establish complete or fresh online values. Cache refresh is separate from stream updates; failures retain the previous snapshot. New features require implementation, historical preparation, and deployment verification.

2.3 Feedback into Serving and Future Training

The Web application reports visible-card exposures, clicks, and structured dislikes through the ingestion API. In cluster mode, Kafka carries events to a Flink or Spark processor, which updates Redis projections and retains history for offline computation. Recent clicks supply retrieval triggers; exposure history and dislike rules support filtering. A dislike can target an item, category, or tags. Behavioral aggregates update user and item feature snapshots; the ranking service must refresh its encoded cache before those updates can affect model inputs.

Retained history supports future recall computation and training. Jobs stage indexes; the console validates and activates serving aliases. Retraining and publication are explicit, not immediate online learning. Graph, recall, and model lifecycles are separate; restoring one does not rewind feedback or the whole system.

3 INTERACTIVE DEMONSTRATION

The demonstration uses OpenRec's existing cluster deployment, launched with the distribution's `example_cluster/start.sh`. This mode connects the recommendation application and management console to online ranking, streaming feedback processing, and offline jobs. A cluster instance is already running; Figure 3 shows captures from the existing interfaces. Participants use an example identity to browse recommendations, inspect the serving

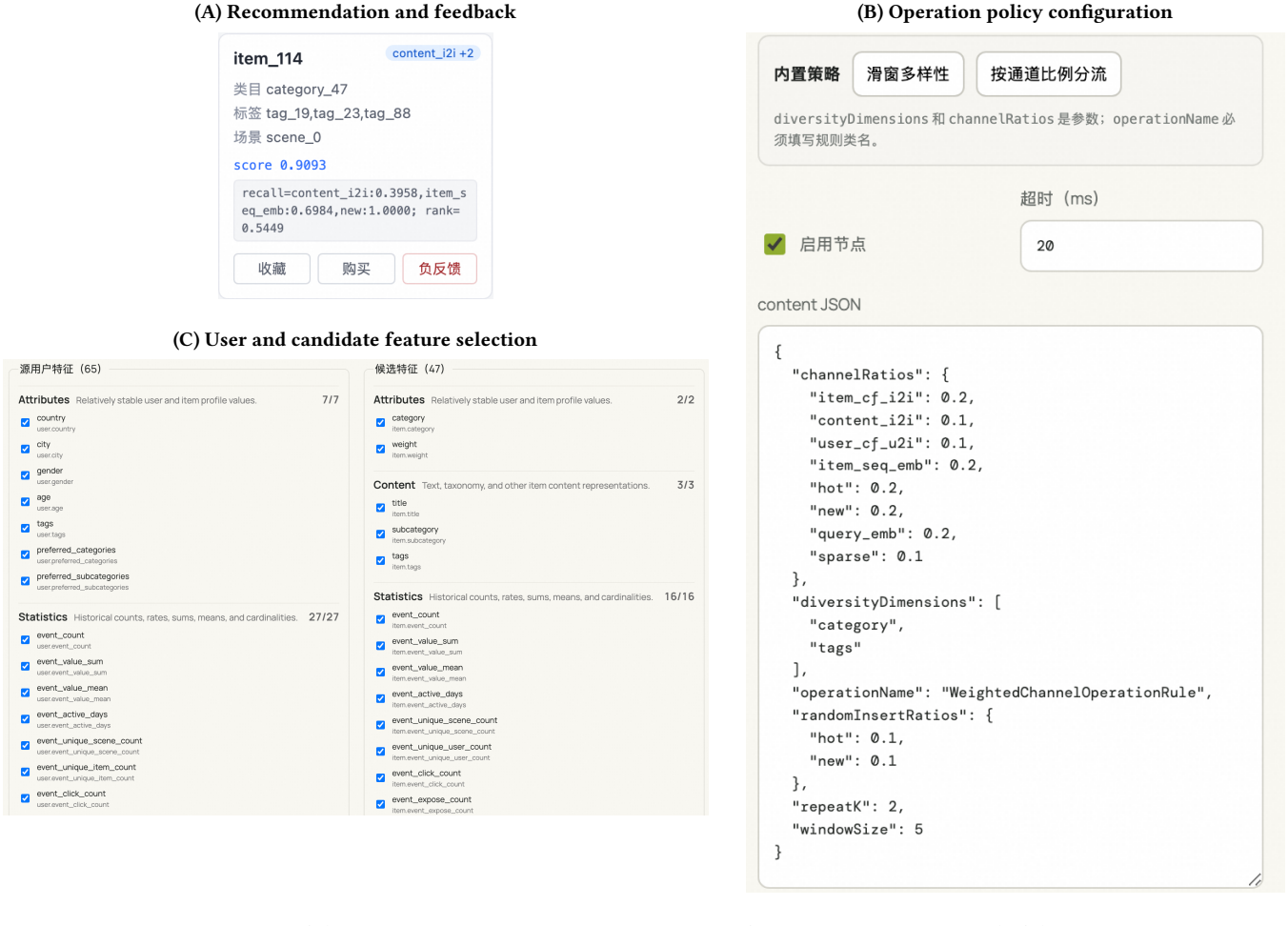


Figure 3: Runtime captures: (A) recommendation scores and feedback (user_0, scene_0, default); (B) operation presets and parameters, with channel allocation selected; (C) user/candidate feature selection by attribute, content, and statistics groups. The training form does not establish a completed run or loaded model.

configuration, and submit feedback. The following walkthrough connects these interactions to the three system capabilities.

3.1 Explore Candidate Supply and List Policies

The recommendation page displays items, channel labels, recall/rank scores, and feedback controls (Figure 3A). The presenter opens the corresponding serving graph in the console and explains how retrieval channels, filters, ranking, and list policies compose the response. Participants can inspect node configuration and compare the channel-allocation and diversity presets (Figure 3B).

To illustrate a configuration change, the presenter disables a contributing recall channel in a named experiment, publishes the graph, and requests recommendations using that experiment. Debug output exposes the channel's execution status and candidate count. The final list may remain unchanged when other channels retrieve the same items. With the operation plugin loaded, participants can also compare result-channel composition, repeated attributes, and list length under different policies. This illustrates configurable behavior rather than a controlled relevance comparison.

3.2 Connect Selected Features to Scoring

The console's training configuration groups user and candidate features by attributes, content, and statistics (Figure 3C). The presenter connects a prepared model release to its selected features, fitted encoding sidecar, input dimension, and offline evaluation. The model-management view and ranking-service state identify the active release; scores on the recommendation page show its use in the serving path.

Participants can inspect these artifacts without running a training job during the session. The presenter can additionally demonstrate activation and restoration of prepared compatible releases. Changing a trained model's feature selection requires retraining; merely editing the training form does not alter the active ranker.

Table 1: KuaiRand-1K temporal holdout results. Learned-model entries are means over three seeds; popularity is deterministic. Full per-run metrics and sample standard deviations are available in the experiment report [3].

	Standard	Random policy	
Model	AUC ↑	AUC ↑	LogLoss ↓
Popularity	0.52956	0.52949	**0.57823**
LR + PIT	0.75142	0.66737	0.68590
FM + PIT	0.75169	0.67316	0.69849
LightGBM + PIT	**0.75410**	**0.67690**	0.71215

3.3 Follow Feedback into Subsequent Requests

Participants submit clicks or dislikes through the recommendation page. The presenter traces feedback through the cluster's streaming path: recent clicks can supply retrieval triggers, while projected dislike rules are consumed by filtering on subsequent requests. An item-level dislike illustrates the distinction between accepting an event and applying its effect after asynchronous projection.

The presenter uses diagnostics to inspect the projected rule and the active filter configuration, alongside the refreshed recommendation list. List disappearance alone does not identify its cause, since exposure filters and other policies also affect membership. Feature updates and ranking-cache refresh are separate steps; feedback does not imply immediate retraining or a guaranteed score change.

On-site deployment. The conference demonstration will use this cluster mode on a prepared host, with browser access to the recommendation page and console. Data, recall indexes, and compatible model releases will be loaded before the session. The presenter will guide configuration changes, restore graph/model versions between sessions, and use separate example identities for feedback interactions. English annotations and a guided walkthrough will accompany the primarily Chinese interface.

4 EVALUATION

Dataset and protocol. We evaluate click prediction using OpenRec's shared point-in-time (PIT) features on KuaiRand-1K [1, 3]. Labels use `is_click`; standard and random-policy test subsets follow `is_rand`. LR, FM, and LightGBM use the same temporal split and 33 logical features: 16 user behavioral statistics, 16 item behavioral statistics, and one scene feature, yielding 46 encoded inputs. The selected catalog features support both online and offline computation. Each learned model is evaluated with seeds 42, 43, and 44. Random-policy observations are excluded from training, validation selection, and feature history; evaluation features are frozen at the training boundary with a one-hour feedback delay. LightGBM uses a binary objective for independent exposure rows.

Results and interpretation. Table 1 compares the three rankers with a popularity baseline. LightGBM achieves the highest AUC under both evaluation policies, while popularity has the lowest random-policy LogLoss. Thus, higher AUC does not imply lower probabilistic prediction loss. The random-policy AUC sample standard deviations are 0.00036, 0.00113, and 0.00079 for LR, FM, and LightGBM, respectively. Compared with the earlier behavior-only configuration in the report, the shared PIT selection increases random-policy AUC by 0.00361 for LR and 0.01393 for FM. These results support the use of shared temporal features and offer concrete model tradeoffs for the scoring demonstration. They do not establish online uplift or verify that an experimental checkpoint and its feature values have been deployed in the demonstration environment.

System verification. The 0.1.0 validation record documents component tests, feature/SDK contract checks, a real-Elasticsearch lifecycle test, and local cluster runs covering ranked requests and streaming checks [4]. These complement the offline results by checking integration behavior. Figure 3 illustrates the running interfaces, while the three demonstration tasks specify observable channel, model, and feedback behavior. We make no end-to-end latency or throughput claim.

5 ETHICAL USE AND AVAILABILITY

We use example identities and fixture data, without collecting personal profiles or conducting a user study. Offline research data retain their providers' access and licensing restrictions and are not redistributed. Captures exclude credentials and sensitive identifiers. Because the console lacks authentication and role-based access control, the venue deployment will be isolated and presenter-supervised.

6 CONCLUSION

OpenRec connects offline recommendation development to Web serving through composable pipelines, shared feature contracts, and feedback processing. The demonstration exposes these connections through candidate supply, list policies, model inputs, and subsequent requests, with explicit publication and bounded restoration.

REFERENCES

[1] Chongming Gao, Shijun Li, Yuan Zhang, Jiawei Chen, Biao Li, Wenqiang Lei, Peng Jiang, and Xiangnan He. 2022. KuaiRand: An Unbiased Sequential Recommendation Dataset with Randomly Exposed Videos. In *Proceedings of CIKM*. ACM, New York, NY, USA, 3953–3957. https://doi.org/10.1145/3511808.3557624

[2] Jianxun Lian, Yuxuan Lei, Xu Huang, Jing Yao, Wei Xu, and Xing Xie. 2024. RecAI: Leveraging Large Language Models for Next-Generation Recommender Systems. In *Companion Proceedings of the ACM Web Conference 2024*. ACM, New York, NY, USA, 1031–1034. https://doi.org/10.1145/3589335.3651242

[3] OpenRec contributors. 2026. KuaiRand-1K OpenRec Global Point-in-Time Features: Local Experiment Report. https://github.com/open-rec/experiments/blob/v0.1.0/results/kuairand-1k-pit-global-v1.md Version 0.1.0.

[4] OpenRec contributors. 2026. OpenRec 0.1.0 Validation Record. https://github.com/open-rec/openrec/blob/v0.1.0/release/VALIDATION.md

[5] Longqi Yang, Eugene Bagdasaryan, Joshua Gruenstein, Cheng-Kang Hsieh, and Deborah Estrin. 2018. OpenRec: A Modular Framework for Extensible and Adaptable Recommendation Algorithms. In *Proceedings of WSDM*. ACM, New York, NY, USA, 664–672. https://doi.org/10.1145/3159652.3159681

[6] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *Proceedings of CIKM*. ACM, New York, NY, USA, 4653–4664. https://doi.org/10.1145/3459637.3482016